

כרטיס ה-LoRa – איפה אנחנו עומדים

רני, 6.10.2026. הקוד ב-github.com/Rani367/e22-lora-arduino. עדכון של המסמך מ-3.9.2026.

מאז הגרסה הקודמת הגיעה הסכמה של הלוח, והספרייה רצה על חומרה אמיתית: שני שני מודולים, PING ו-PONG בין שניהם, וסדרת בדיקות מלאה. על לוח הטיסה עצמו היא עוד לא רצה. המסמך הזה מסכם מה אנחנו יודעים עכשיו ומה הצעד הבא.

מה הפרויקט בעצם

על הלוויין יש כרטיס תקשורת עם מודול LoRa של Ebyte. ה-OBC לא מדבר איתו, רק מדליק ומכבה, אז הכרטיס צריך להסתדר לבד: כל כמה זמן לשדר ביקון עם מונה, ובין לבין להאזין ולשדר חזרה מה שנקלט. אנחנו כותבים את הקוד שרץ עליו, בסביבת ארדואינו.

חילקנו את זה לשניים. החלק הראשון הוא ספרייה שיודעת לדבר עם המודול. החלק השני הוא הלוגיקה של הלוויין (ביקון, משיב, מעבר ביניהם) שתשתמש בספרייה. החלק הראשון כתוב ונבדק על השולחן עם Arduino Uno. החלק השני עוד לא התחיל.

מה למדנו על המודול

המסמך המקורי של הפרויקט מדבר על פני SPI, NSS, DIO0 וכו'. זה לא רלוונטי למודול שיש לנו. ה-E22 הוא לא שבב LoRa חשוף אלא לוח קטן עם מיקרו-בקר משלו, ומדברים איתו ב-UART (9600 כברירת מחדל). חוץ מה-UART יש שלושה פינים שחשובים: M0 ו-M1 בוחרים את מצב העבודה, ו-AUX אומר אם המודול עסוק (LOW) או פנוי (HIGH). עוד כמה דברים שכדאי לדעת לפני שמתכננים משהו:

- אי אפשר לבחור Spreading Factor. מה שיש זה "קצב אוויר" בין 2.4 ל-62.5 kbps, וכל קצב הוא איזה צירוף SF/BW ש-Ebyte החליטו עליו. הניסוי של "לשנות SF כל 10 שניות" מהמסמך הופך לשינוי קצב אוויר. אותו רעיון.
- הצד השני של הקישור כנראה צריך להיות גם Ebyte. E22 עוטפים את ה-LoRa בפרוטוקול משלהם עם FEC, ולא מצאנו אף אחד שמפענח את זה עם SX127X רגיל. דוד אומר שקלט פעם עם משהו אחר, אז נבדוק במקום לנחש. בכל מקרה הכרטיס במקור נועד לתקשורת בין לוויינים עם אותו כרטיס.
- התדר הוא מספר ערוץ: 410.125 MHz ועוד ערוץ במגה-הרץ, ערוצים 0 עד 83. ברירת המחדל היא ערוץ 23, כלומר 433.125. הלוויין הראשי על 436.4, אז ערוצים 25 עד 28 מחוץ לתחום. התדר הוא פרמטר ולא קבוע בקוד.
- המודול לא שומר על גבולות של פקטות בצד ה-UART. שתי פקטות שמגיעות ברצף יכולות לצאת כזרם אחד. פורמט הפקטה שלנו יצטרך בייטים של סנכרון, אורך ו-checksum. לדעתי זו ההחלטה הכי חשובה בחלק השני.
- **חדש:** כשבייט ה-RSSI מופעל, המודול מוסיף אותו אחרי כל פקטת רדיו, לא פעם אחת בסוף ההודעה. הודעה של 300 בייט מגיעה כ-240 בייט, RSSI, ועוד 60 בייט ובייט RSSI. בייט RSSI של 0 הוא לא מדידה תקפה. גם את זה פורמט הפקטה צריך לקחת בחשבון.
- **חדש:** המודולים מגיעים מהמפעל עם הרגיסטרים 00 00 83 17 00 62 00 00. רגיסטר 6 הוא 0x83 ולא 0x03 כמו במדריך, כלומר בייט ה-RSSI מופעל מהמפעל.
- **חדש:** בקושחה 0-21-AUX, 7453 יורד ל-LOW רק לכמה מילישניות אחרי שליחה, ולא לכל זמן השידור באוויר. אי אפשר להשתמש בו כדי לדעת מתי הפקטה באמת שודרה.

- **חדש:** תצורה זמנית (`persist=false`) נשמרת ב-RAM של המודול. איפוס של הבקר לא מוחק אותה, רק כיבוי של המודול.

מה אנחנו יודעים על הלוח

נושא	מה ידוע
מודול	E22-400T30D, מסומן V2. גרסת ה-DIP של 1 וואט: 7 פינים, אין RESET, דורש 5V ומושך עד 620mA בשידור. בשני המודולים שבדקנו הקושחה היא 7453-0-21.
בקר	ATmega328PB, לוגיקה של 5V, 2KB RAM, 32KB. בלי USB על הלוח.
פינים	מהסכמה ("ISC", "KiCad"): המודול בשקע J3. TXD של המודול נכנס ל-PD2, ו-RXD מקבל מ-PD3. אלה לא UART חומרתי, אז בלוח הטיסה הרדיו על SoftwareSerial. M0 מגיע מ-PB0 (D8), ו-M1 מ-PE3 (D26). הדוגמאות כבר מעודכנות לפינים האלה.
מתחים	רק על RXD יש מחלק מתח: R5 של 1.3k ו-R4 של 2.7k, שנותנים בערך 3.4V. M0 ו-M1 מקבלים 5V ישירות מהבקר. על השולחן זה עבד.
AUX	לא מחובר לבקר. הוא הולך למעגל של מגבר ושני טרנזיסטורים (Q1, Q2, C5, U4A) שמושך את ה-RESET. בלי AUX הספרייה מחכה זמנים קבועים במקום לחכות ל-AUX. את המצב הזה עוד לא בדקנו על חומרה.
watchdog	ב-(U4) MC1458 מוצב רק חצי A, ולפיני ההזנה שלו (4 ו-8) אין net ואין מסלולים, אז כמו שהוא מצויר הוא לא יכול לעבוד. דוד: להשאיר את זה לסוף.
צריבה	J1 הוא מחבר בסגנון FTDI על USART0, עם קבל על DTR. J5 הוא מחבר ISP. פין 5 של J5, שמסומן +5V, לא מחובר ל-VCC בסכמה. עוד לא ניסינו לצרוב את הלוח.
הזנה	מ-(OUT4..6_5V0) PC104 דרך נגדי 0 אוהם. קווי הנתונים של ה-PC104 לא מחוברים.
תדר	פרמטר. הלוויין על 436.4. רישוי לא נדרש במעבדה בהספקים האלה.
תחנת קרקע	עוד פתוח: האם מקלט LoRa רגיל יכול לפענח את המסגור של Ebyte. נבדוק.

מה נבדק על השולחן

ב-6.10 חיברנו שני Arduino Uno לאותו מחשב, כל אחד עם E22-400T30D על SoftwareSerial. החיבורים מצד המודול: D6 → AUX, D5 ← M1, D4 ← M0, D11 ← RXD, D10 → TXD. הכול בתצורה זמנית, ערוץ 23, עד 21 dBm. מה עבר:

- קריאה וכתביה של התצורה עם קריאה חוזרת, כל פונקציות ה-`set...`, פקודות AT. המודול מזדהה כ-`DEVTYPE=E22-400T30D`.
- PingPong: כל PING קיבל PONG.
- כל ששת קצבי האוויר בשני הכיוונים: 120 מתוך 120 פקטות הגיעו.
- קצב אוויר, ערוץ או מזהה רשת שונים בשני המודולים: לא מתקבל כלום, כמו שצריך.
- כתובות: כתובת שונה לא מקבלת כלום. שליחה מ-0xFFFF או האזנה על 0xFFFF מקבלת הכול.

- שליחה עם כתובת יעד (`sendTo`): מגיעה בלי שלושת בייטי הכותרת. כתובת או ערוץ שגויים לא מקבלים כלום.
- הודעה של 300 בייט, מחולקת לפקטות של 240 ושל 32 בייט.
- במצבי שינה ותצורה לא מתקבל כלום. `send()` מסרבת לשלוח במצב שינה וב-`wake-on-radio`.
- UART ב-19200 וב-38400, כולל המעבר ל-9600 בשביל תצורה.
- רעש רקע: בין 78 dBm ל-82 dBm. זמנים: החלפת מצב כ-22ms, קריאת תצורה כ-70ms, כתיבה עם בדיקה כ-100ms.

נמצאו ותוקנו שני באגים: `atCommand()` החזירה הצלחה כשהמודול דחה פקודה (`FF FF FF`), ובייט RSSI של 0 הודפס כ-0 dBm. ה-RSSI כשהמודולים צמודים נראה רווי (בין 0 dBm ל-7 dBm-), צריך למדוד שוב במרחק של כמה מטרים.

עוד לא נבדק: לוח הטיסה עצמו (בלי `SoftwareSerial`, `AUX` על D2/D3, `ESP32`, `HardwareSerial`, שמירת תצורה בפלאש (`persist=true`), `timeouts` כשהמודול מנותק, קבלה ב-`relay`, `LBT`, `wake-on-radio`, והספק מעל 21 dBm. גם יציאת ה-RF עוד לא נמדדה בספקטרום אנלייזר.

מה יש בריפו

`src/E22.cpp` ו-`src/E22.h` זו הספרייה. היא יודעת לעבור בין מצבים עם M0/M1 ולחכות ל-AUX, לקרוא ולכתוב את 9 רגיסטרי התצורה (ואחרי כל כתיבה לקרוא חזרה ולוודא), לשלוח פקודות AT, ולהעביר בייטים הלוח וחזור. היא מקבלת `HardwareSerial` או `SoftwareSerial`. בלי תלויות, ומספיק קטנה ל-2KB. כל המתנה בה היא עם `timeout`, כי אם משהו נתקע על הלויין אף אחד לא יבוא לשחרר.

יש שלוש דוגמאות ב- `examples/`:

- `ReadConfig` קורא את הרגיסטרים וגרסת הקושחה ומדפיס. זה מה שמריצים ראשון על לוח חדש.
 - `PingPong` לשני לוחות, אחד שולח PING והשני עונה PONG, עם RSSI.
 - `AirRateSweep` הניסוי מהמסמך: שני הלוחות עוברים על ששת הקצבים, 10 שניות כל אחד.
- ב- `docs/` יש את המדריך של המודול, את המסמך המקורי ואת המסמך הזה. יש גם CI שמקמפל כל דוגמה ל-`ATmega328PB`, ל-`Uno` ול-`ESP32` בכל `push`.

איך מריצים

```
brew install arduino-cli
arduino-cli config init
arduino-cli config add board_manager.additional_urls \
  https://mcudude.github.io/MiniCore/package_MCUdude_MiniCore_index.json
arduino-cli core update-index
arduino-cli core install MiniCore:avr arduino:avr
git clone https://github.com/Rani367/e22-lora-arduino.git
cd e22-lora-arduino
arduino-cli compile --fqbn MiniCore:avr:328:variant=modelPB,clock=16MHz_external \
  --library . examples/ReadConfig
arduino-cli compile --fqbn arduino:avr:uno --library . examples/ReadConfig
```

לצרום ל-`Uno` ולבחור תפקיד בלי לערוך את הקוד (שולח 1, מקבל 0):

```
arduino-cli board list
arduino-cli compile -u -p /dev/cu.usbmodemXXXX --fqbn arduino:avr:uno --library . \
  --build-property "compiler.cpp.extra_flags=-DROLE_SENDER=0" examples/PingPong
```

אם מישהו מעדיף ESP32 על השולחן (הורדה של בערך 2GB):

```
arduino-cli config add board_manager.additional_urls \
  https://espressif.github.io/arduino-esp32/package_esp32_index.json
arduino-cli core install esp32:esp32
```

אפשר גם ב-Arduino IDE, פשוט להעתיק את התיקיה ל- ~/Documents/Arduino/libraries/.
בכל דוגמה יש בלוק בהתחלה שבוחר פורט ופינים לפי הלוח: Uno או ESP32.

חיווט למודול

פינ במודול	מה זה	לוח הטיסה	Uno על השולחן	הערות
1 M0	מצב, ביט 0	D8 (PB0)	D4	יציאה מהבקר. לא להשאיר צף.
2 M1	מצב, ביט 1	D26 (PE3)	D5	אותו דבר.
3 RXD	נתונים אל המודול	D3 (PD3), דרך מחלק	D11, דרך מחלק	כניסה של 3.3V. מחלק של 1.3k בטור ו-2.7k לאדמה, כמו בלוח.
4 TXD	נתונים מהמודול	D2 (PD2)	D10	
5 AUX	עסוק/פנוי	לא מחובר לבקר	D6	רק לקרוא. אם מושכים אותו ל-LOW בזמן הדלקה המודול נכנס למצב עדכון קושחה ומפסיק לדבר.
6 VCC	הזנה	5V	5V	עד 620mA בשידור. אם ההזנה צונחת המודול מתאפס באמצע פקטה.
7 GND	אדמה	GND	GND	

על ה-Uno: לא לחבר שום דבר ל-D0 ו-D1. זה קו הצריבה, ומודול שמחובר שם חוסם צריבה, עם השגיאה not in sync: resp=0x0a. לחבר אנטנה לפני שמדליקים. ב-21 dBm ה-5V של ה-Uno מספיק, עם קבל של 100 עד 470µF ליד המודול. מעל זה צריך ספק 5V נפרד של לפחות 1A, עם אדמה משותפת.
ההצלחה הראשונה נראית ככה: ReadConfig מדפיס ערוץ 23 (433.125 MHz), קצב 2.4 kbps, הספק 0 level, וגרסת קושחה 7453-0-21. זה מה שהמודול יוצא איתו מהמפעל.

הצעה לחלוקת עבודה

שלושה כיוונים שכמעט לא תלויים אחד בשני. מי לוקח מה, נחליט ביחד.

- 1. **חומרה.** להבין איך צורבים את לוח הטיסה, ולהריץ עליו ReadConfig ואז PingPong מול Uno. זו הפעם הראשונה שהספרייה תרוץ בלי AUX. לבדוק בספקטרום אנלייזר במעבדה שהמודול באמת משדר (PingPong משדר כל 2

שניות על 433.125 MHz). ה-watchdog בסוף.

2. **החלק השני.** להגדיר את פורמט הפקטה (סנכרון, סוג, מונה, זמן פעולה, קצב, אורך, CRC, payload), לכתוב את מכונת המצבים ביקון/משיב על `millis()` ו-watchdog, ולשמור את המונה ב-EEPROM כדי שישרוד כיבוי. זה sketch נפרד שמשתמש בספרייה. לזכור את בייט ה-RSSI שמגיע אחרי כל פקטת רדיו.
3. **הספרייה והניסוי.** להריץ את AirRateSweep במרחק של כמה מטרים לפחות ולרשום RSSI ואחוזי קליטה לכל קצב. לבדוק את מה שעוד לא נבדק (שמירה בפלאש, ESP32, timeouts). אולי לשאול את Ebyte על הפקודה AT+ULORAM (שליטה ישירה ב-SF/BW, קיימת אבל לא מתועדת).

כמה כללים

- עובדים ב-branch ופותרים PR ל-main.
- לפני push מקמפלים לפחות דוגמה אחת. ה-CI יתפוס את השאר.
- מה שלא רץ על חומרה מסומן ככה ב-README. כשמשוו עובד באמת, מעדכנים.
- לא נוגעים בערוצים 25 עד 28.
- בניסויים משנים תצורה עם `persist=false` כדי לא לשחוק את הפלאש של המודול.
- על השולחן משדרים ב-21 dBm. שני מודולים צמודים ב-30 dBm מציפים אחד את השני.
- הספרייה רק מזיזה בייטים. פורמט, checksum ומונים נשארים בסקצ' שמעליה, ככה אותה ספרייה תשמש גם את תחנת הקרקע.
- 2KB RAM. מחרוזות בתוך `F()`, באפרים קטנים, בלי `printf`.

מה עושים עכשיו

1. סכמה מריף, לעדכן את הפינים בדוגמאות. נעשה.
2. ReadConfig על מודול אמיתי, תמונת המפעל וגרסת הקושחה. נעשה.
3. PingPong בין שני מודולים. נעשה, על שני Uno.
4. בדיקה בספקטרום אנלייזר במעבדה.
5. צריבה של לוח הטיסה ו-ReadConfig עליו.
6. AirRateSweep במרחק, עם רישום RSSI.
7. רק אז החלק השני. ה-watchdog בסוף.